Activation Steering for Smart Contract Vulnerability Detection: A Calibration Mechanism Bounded by Pre-Training Knowledge

Abhirup Mukherjee
The University of Texas at Dallas
axm240026@utdallas.edu

Latifur Khan
The University of Texas at Dallas
lkhan@utdallas.edu

Bhavani Thuraisingham
The University of Texas at Dallas
bxt043000@utdallas.edu

IEEE International Conference on Intelligence and Security Informatics (ISI) 2026

Outline

- Problem
- Background
- Methodology
- Experimental Setup
- Results
- Contributions, limitations, and future work

Smart Contract Exploits Cost Billions

- A smart contract is a self-executing, high-level program written in the Solidity language that resides at a specific address on the Ethereum blockchain.
- Exploits in these contracts have incurred total losses upwards of $16B[1] in DeFi (decentralized finance).
- For eg, DAO (2016) (~$60 M), Parity Wallet (2017) (~$150M), Ronin Bridge (2022) (~$620M), Wormhole (2022) (~$320M), Euler Finance (2023) (~$200M).

1. "DEFI Hacks & Exploits Database - DefiLlama," https://defillama.com/hacks

What Is Solidity?

- It is the dominant smart-contract language, running on the Ethereum Virtual Machine (EVM).
- The EVM is a 256-bit stack machine where every opcode costs gas and has a globally consistent state[1].
- Solidity compiles to EVM bytecode; once deployed, that bytecode is immutable[2].

1. Wood, G. (2014). Ethereum Project Yellow Paper. https://ethereum.github.io/yellowpaper/paper.pdf
2. Upgrading smart contracts. https://ethereum.org/developers/docs/smart-contracts/upgrading/
3. Picture: https://www.soliditylang.org/
4. Picture: https://ethereum.org/

Why Solidity is not regular code

Solidity presents its own unique set of challenges:

- Vulnerability classes do not transfer from C / C++. They are unique to this language.
- Blockchain has its own native failure modes. E.g., reentrancy has no C/C++ analogous vulnerability.
- A missed vulnerability cannot be patched after deployment.

Our Goal: Explore light-weight intervention methods for a specialized classification task

Activation steering sits between prompting (essentially free) and fine-tuning (expensive) in terms of compute costs

Zero-shot	Few-shot	Steering	PEFT (LoRA)	Full fine-tuning
No examples No cost	2-3 examples No gradients	Single direction vector No gradients	Reduced weights Gradients required	All weights Many GPU-hours

low cost → high cost

- Full fine-tuning must hold weights, gradients, and optimizer states in memory, which requires roughly 8x the memory required for inference.
- LoRA reduces the optimizer state in memory, but not the backward pass since gradients must still flow through every frozen layer.
- Activation steering computes no gradients at all, since a single offline forward pass yields one direction vector that costs only a vector addition to apply.

What Is Activation Steering?

Activation steering is an inference-time technique that modifies a Large Language Model's (LLM) internal hidden states (activations) to control its behavior without retraining or fine-tuning.

$$\tilde{h}^{\ell} = h^{\ell} + \alpha \cdot v^{\ell}$$

- $\tilde{h}^{\ell}$: Updated hidden state activation at layer l
- h^{ℓ}: hidden state activation at layer l
- α: scalar magnitude
- v^{ℓ}: direction vector

1. A. Zou et al., "Representation Engineering: A Top-Down Approach to AI Transparency," arXiv:2310.01405, 2023
2. N. Rimsky et al., "Steering Llama 2 via Contrastive Activation Addition," ACL 2024

Prior work in smart contract vulnerability detection

Paper	Contribution
F. Mi, Z. Wang, C. Zhao, J. Guo, F. Ahmed and L. Khan, "VSCL: Automating Vulnerability Detection in Smart Contracts with Deep Learning," 2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), Sydney, Australia	Bytecode-based deep feature generator for smart-contract vulnerability classification
Sakib, M. N. and L. Khan, "VulnPatch: Multi-Agent Automated Smart Contract Vulnerability Detection, Explanation, and Mitigation Framework," 2025 7th International Conference on Blockchain Computing and Applications (BCCA), Durbovnic, Croatia	Three QLoRA-fine-tuned LLM agents (VulnHunter, VulnClassifier, VulnMedic) for end-to-end Solidity detection + patching

Prior Work in activation steering and LLM-based vulnerability detection

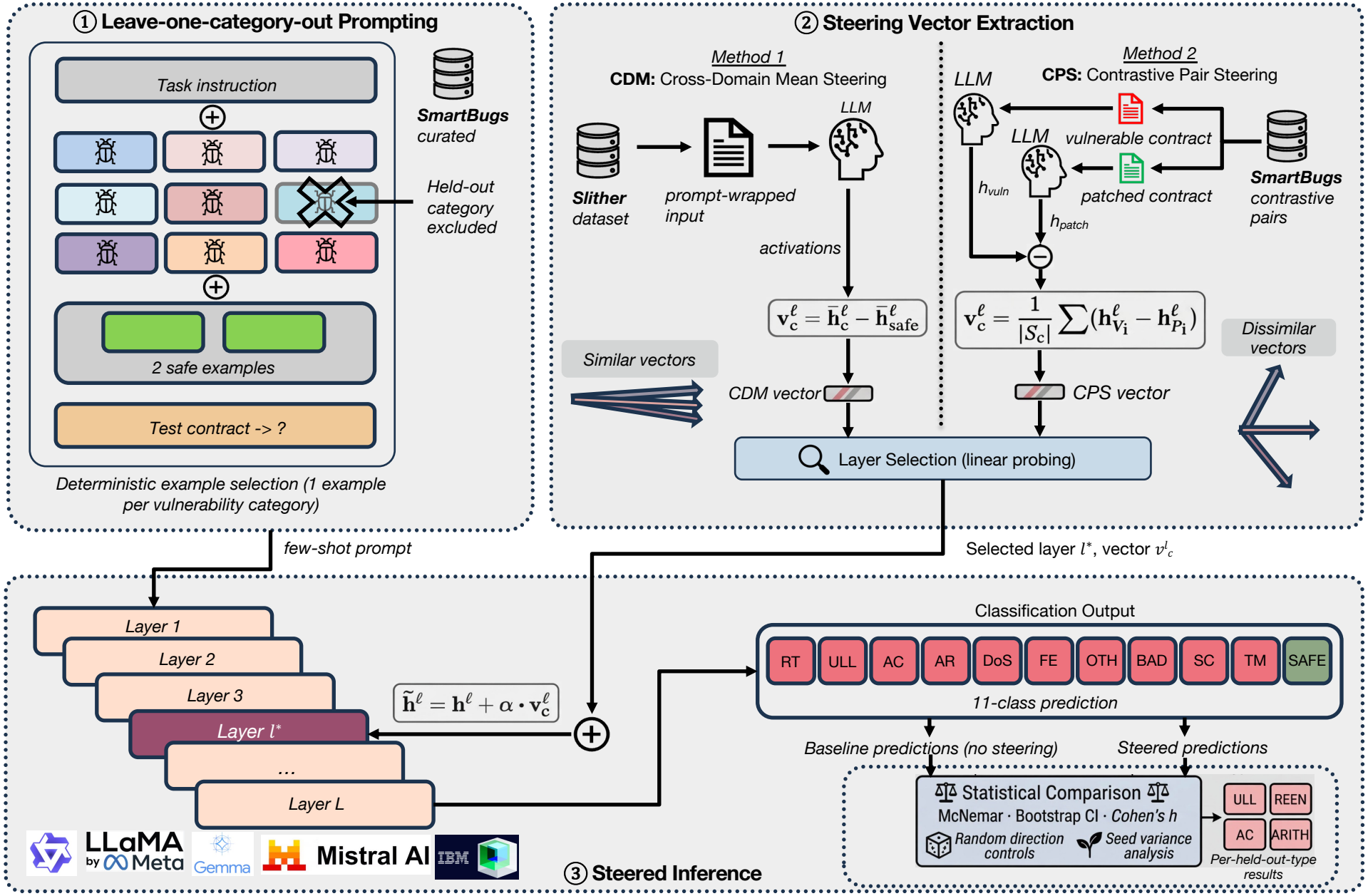
Paper	Contribution
A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski, S. Goel, N. Li, M. J. Byun, Z. Wang, A. Mallen, S. Basart, S. Koyejo, D. Song, M. Fredrikson, J. Z. Kolter, and D. Hendrycks, "Representation Engineering: A Top-Down Approach to AI Transparency," arXiv:2310.01405, 2023	Concepts lie along linear directions in activation space
N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. Turner, "Steering Llama 2 via Contrastive Activation Addition, " ACL 2024	Multi-pair difference-in-means
J. Li, L. Cui, J. Zhang, H. Fei, Y. Chen, and H. Zhu, "Steering Large Language Models for Vulnerability Detection," IEEE ICASSP 2025	Only prior steering work on code vulnerability (C/C++, +16% F1)
Yu, L., Z. Huang, H. Yuan, S. Cheng, L. Yang, F. Zhang, C. Shen, J. Ma, J. Zhang, J. Lu, and C. Zuo, "Smart-LLaMA-DPO: Reinforced Large Language Model for Explainable Smart Contract Vulnerability Detection," arXiv:2506.18245, 2025	Large fine-tuned pipelines; computationally expensive

Our motivation: we are looking for interpretable, lightweight interventions that use the model's own internal representations without fine-tuning

Approach	Examples	Strengths	Drawbacks
Static Analysis Tools	Slither[1], Mythril, Oyente	Precise	Inflexible rules
Lightweight Prompting	Zero-shot / few-shot	Cheap, flexible	Poor performance
Fine-Tuned LLMs	VulnPatch, Smart-LLaMA-DPO[2]	High F1 scores	Computationally expensive

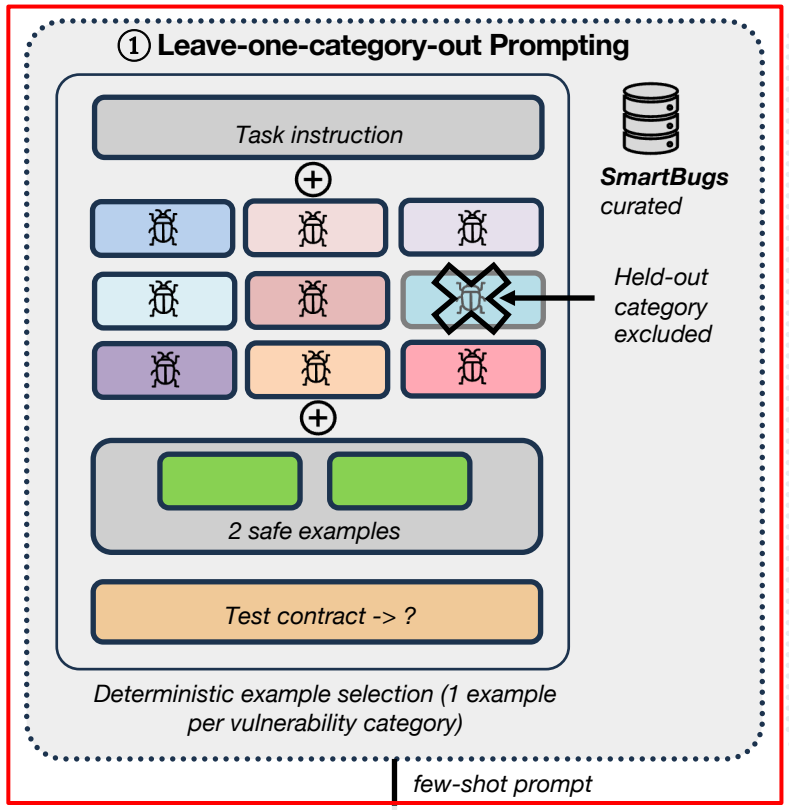
1. J. Feist et. al., "Slither: A Static Analysis Framework for Smart Contracts," IEEE/ACM WETSEB
2. L. Yu et al., "Smart-LLaMA-DPO: Reinforced Large Language Model for Explainable Smart Contract Vulnerability Detection," arXiv:2506.18245, 2025

Our Methodology: An Overview



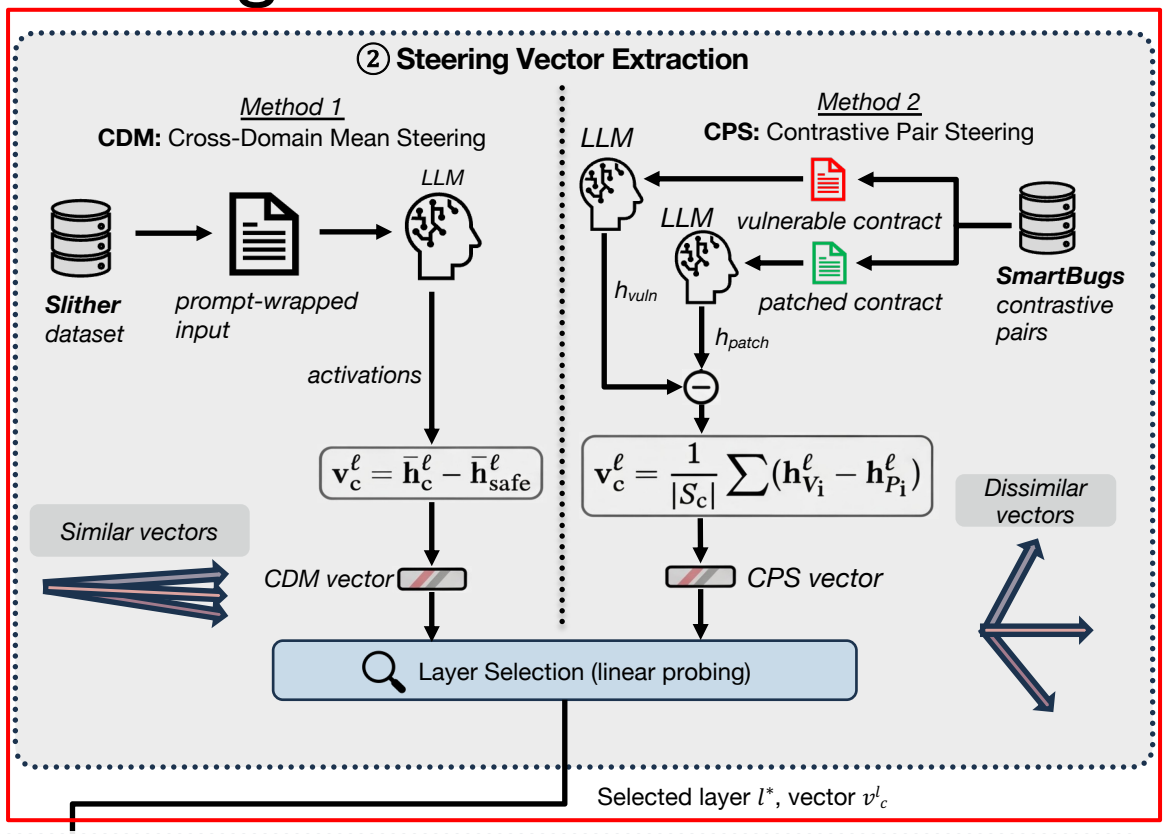
Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

Phase 1: Build the prompt



Phase 1 constructs a leave-one-out few-shot prompt.

Phase 2: Extract steering vectors and select best hidden layer

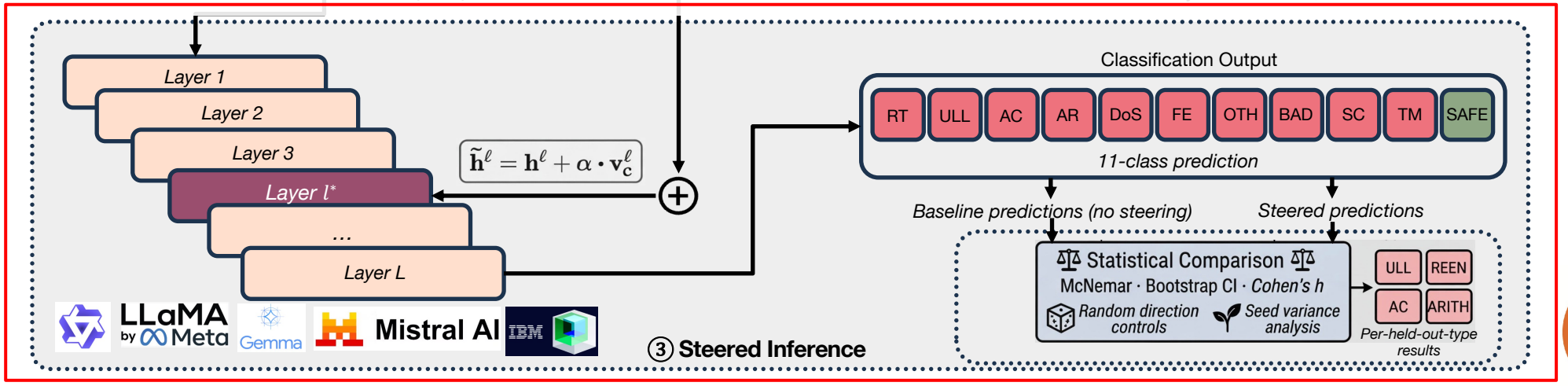


Phase 2 extracts steering vectors via CDM or CPS and selects the best layer based on its AUROC curve

Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

Phase 3: Inject vector into target layer

Phase 3 injects the vector via a PyTorch forward hook during inference.



We prepare the steering vector in two ways: CDM and CPS

CDM: Cross-Domain Means

difference-in-means b/w the two groups of activations[1]

$$v_c^\ell = \bar{h}_c^\ell - \bar{h}_{\text{safe}}^\ell = \frac{1}{|S_c|} \sum_{i \in S_c} h_i^\ell - \frac{1}{|S_{\text{safe}}|} \sum_{j \in S_{\text{safe}}} h_j^\ell \qquad (2)$$

CPS: Contrastive Pair Steering

paired per-sample subtraction of activations[2]

$$D_i^\ell = h_{V_i}^\ell - h_{P_i}^\ell \qquad (3)$$

where $h_{V_i}^\ell$ and $h_{P_i}^\ell$ refer to activations of a vulnerable contract and its minimally-patched safe counterpart. The category steering vector is:

$$v_c^\ell = \frac{1}{|S_c|} \sum_{i \in S_c} D_i^\ell \qquad (4)$$

$$\tilde{h}^l = h^l + \alpha \cdot v^l \qquad (5)$$

Updated hidden state activation at layer l
$\tilde{h}^l \in \mathbb{R}^{B \times T \times d}$

hidden state activation at layer l
$h^l \in \mathbb{R}^{B \times T \times d}$

scalar magnitude
$\alpha \in \mathbb{R}$

direction vector
$v^\ell \in \mathbb{R}^d$

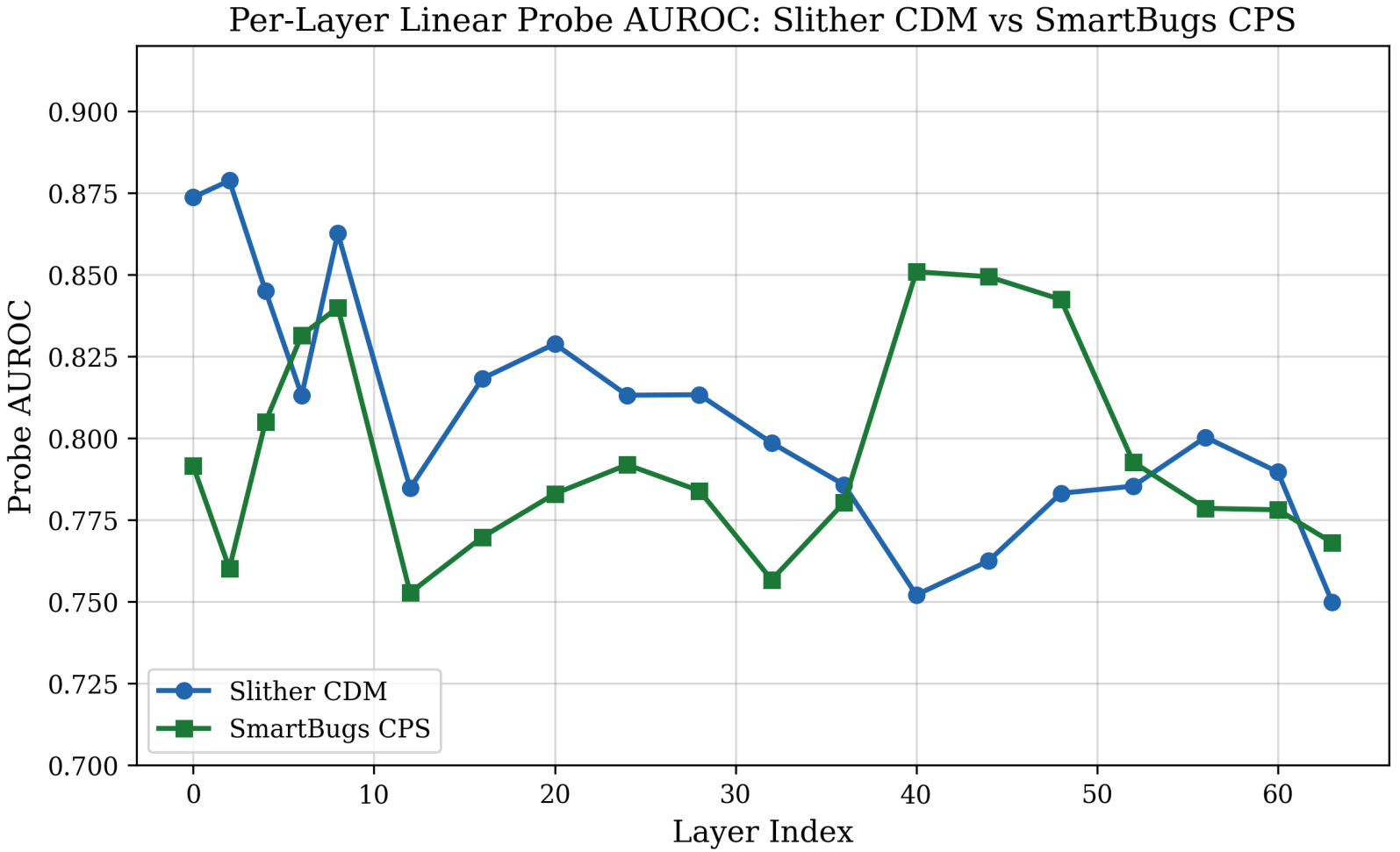
Paper Eqs. 2, 3, 4, 5

1. A. Zou et al "difference-in-means"
2. N. Rimsky et al. "contrastive activation addition"

Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

How to select the best layer? Linear probe AUROC by layer gives us our candidate layer (where AUROC is highest)

- Each point is the accuracy of a linear probe trained on that layer's hidden states to separate vulnerable contracts from safe ones; the higher the value, the more readable the vulnerability signal is at that depth.
- The blue curve (CDM) is highest in the early layers. Since CDM draws its two groups from entirely different contracts, the early layers can already separate the signal based on surface features like variable names and boilerplate code, before the model interprets code semantics.
- The green curve (CPS) is uneven through the early and middle layers and peaks b/w layers 40 to 50. Here, since each pair consists of a vulnerable contract and its minimally patched safe counterpart, surface features give nothing away; the model must interpret code semantics before the two can be separated.



Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

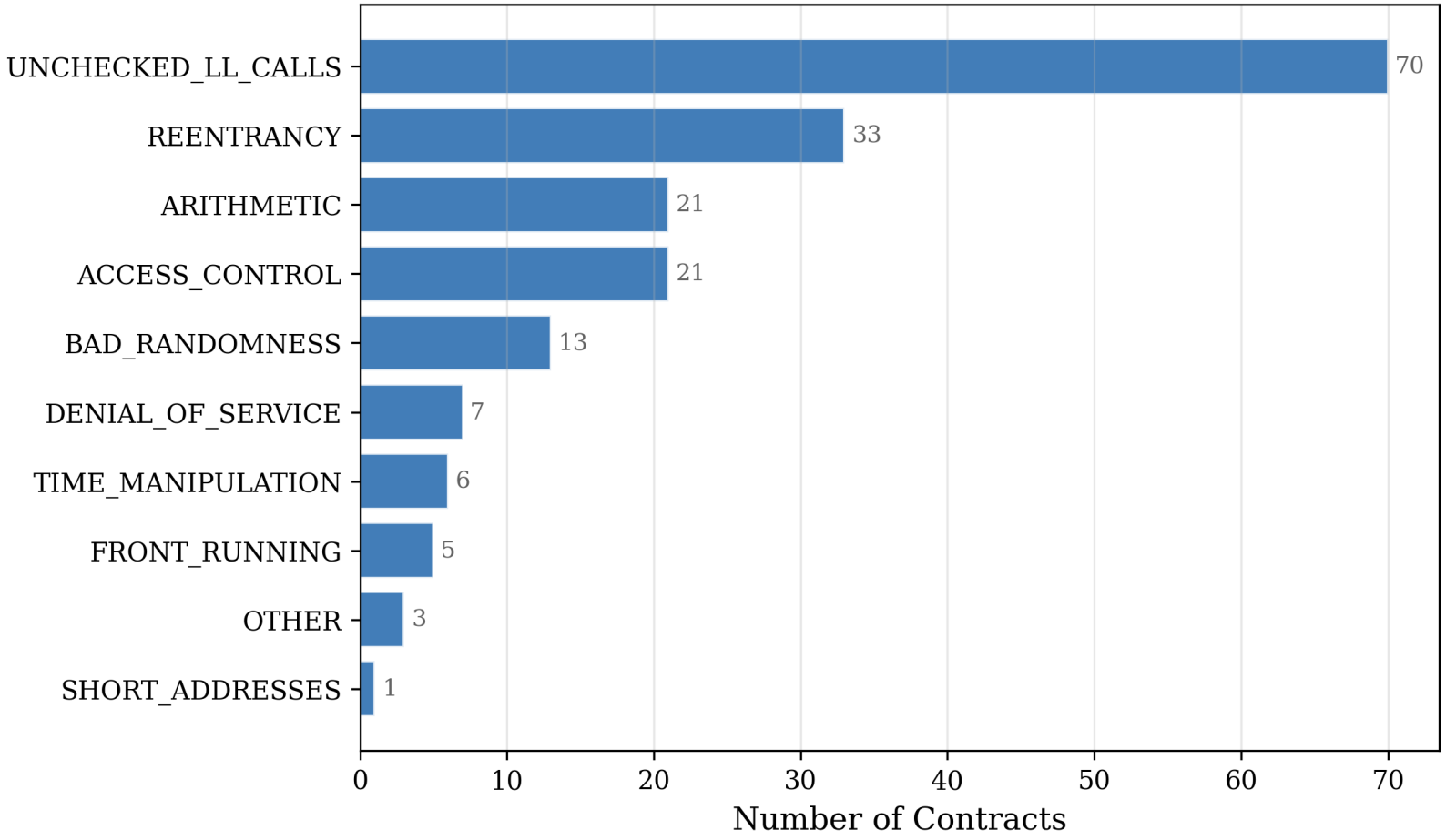
DASP Top 10: The 11-Class Label Space

Category	Defect pattern
Reentrancy (REEN)	External call re-enters before state update
Access Control (AC)	Missing owner / role check; tx.origin misuse
Arithmetic (AR)	Integer overflow / underflow without SafeMath
Unchecked LL Calls (ULL)	.call / .send return value ignored
Denial of Service (DoS)	Unbounded loops; gas exhaustion
Bad Randomness (BAD)	block.blockhash / block.timestamp as RNG
Front-Running (FR)	Tx ordering exploitable; missing commit-reveal
Time Manipulation (TIME)	block.timestamp as authoritative time
Short Addresses (SC)	ABI encoding loses bytes for 20-byte addr
Other (OTH)	Uninitialized storage, unreachable code, etc.
SAFE	Safe contract

4 held-out categories that we test on

Decentralized Application Security Project (DASP) Top 10 (NCC Group, 2018)

The long-tail distribution of our test set motivated our leave-one-category-out evaluation on the 4 largest categories



Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

Prompt structure: Leave-one-category-out; the model never sees the vulnerability type it must classify

````
Example 1 (ACCESS_CONTROL):
```solidity
function withdraw(uint _amount) public {
    msg.sender.transfer(_amount);  // missing onlyOwner
}
```
Classification: ACCESS_CONTROL

.
.
.

Example 9...

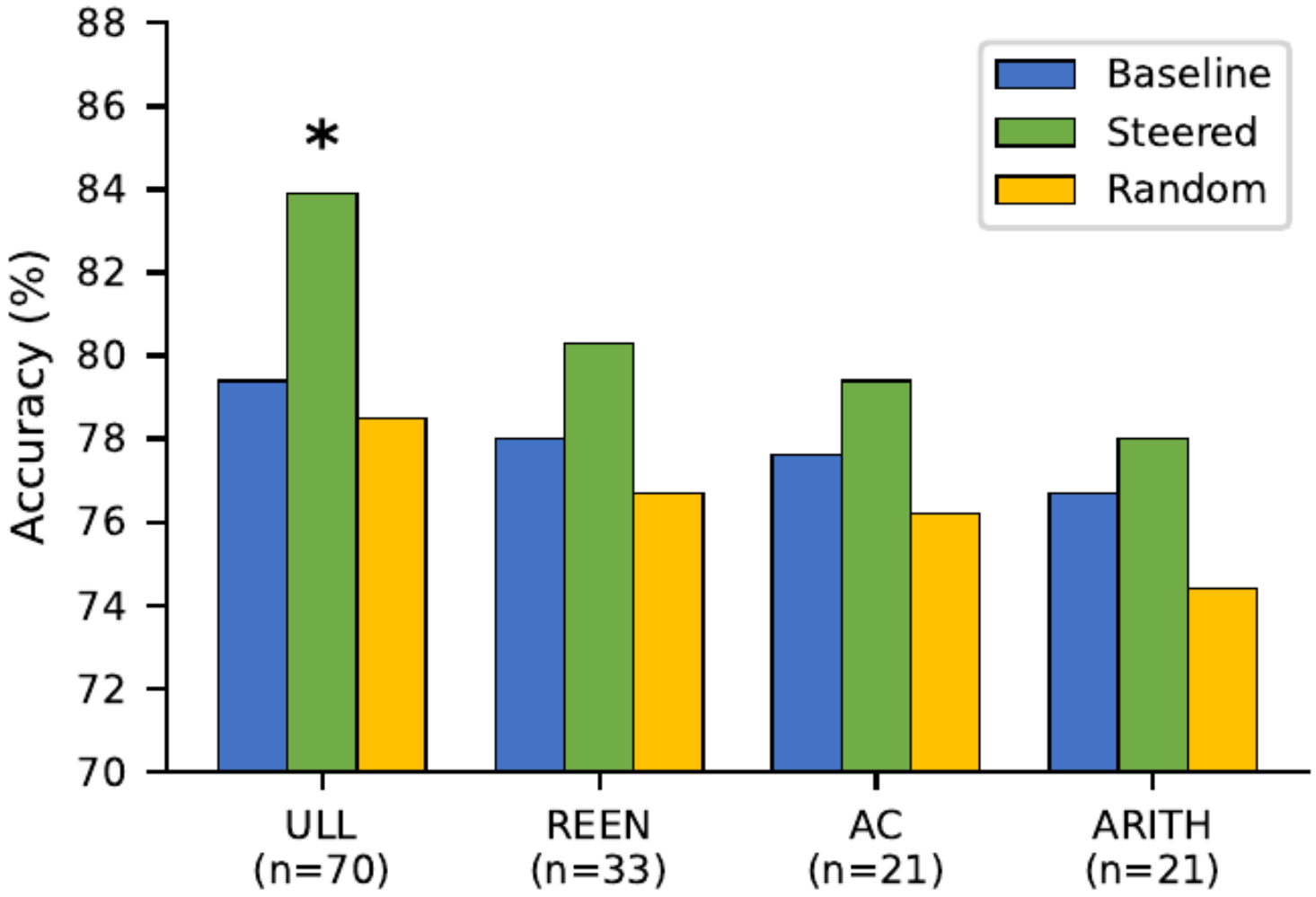
Example 10 (SAFE):
```solidity
function withdraw(uint _amount) public onlyOwner {
    require(address(this).balance >= _amount);
    payable(msg.sender).transfer(_amount);
}
```
Classification: SAFE

Now analyze this contract:
```solidity  <test contract>  ```
Classify as one of: ACCESS_CONTROL, ARITHMETIC,...,TIME_MANIPULATION, UNCHECKED_LL_CALLS, SAFE

Your classification:
````

- The model's prompt consists of the following: one contract from each DASP category (9) (minus the held-out category) + 2 SAFE contracts + the test contract
- Eg, if REENTRANCY is held-out, the model is not shown an example of this vulnerability
````

We see a +4.5 pp improvement on the Unchecked Low-Level Calls (ULL) vulnerability



ULL (held-out)

- +4.5 pp accuracy jump from 79.4% to 83.9%
- p = 0.006 (McNemar)
- ULL recall went from 80% to 87%

Fig. 2. Qwen2.5-Coder-32B, CDM universal vector, layer 16, α = 2.0. Best configuration selected post-hoc on test-set accuracy; p is nominal and not corrected for multiple comparisons

Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

Steering results across all four held-out types

TABLE I

STEERING RESULTS: BASELINE VS. BEST STEERED VS. RANDOM DIRECTION (QWEN + CDM, $n = 223$). ACC: OVERALL ACCURACY (%). F1: HELD-OUT-TYPE F1. STEERED: BEST CONFIG PER TYPE. RANDOM: L16 α=2.0 FOR ALL TYPES.

	Acc (%)			F1				
HO	Base	Steer	Rand	Base	Steer	Rand	Δ_S	p
ULL	79.4	**83.9**	78.5	.855	**.904**	.837	**+4.5**	**.006**
REEN	78.0	80.3	76.7	.741	.811	.779	+2.2	.125
AC	77.6	79.4	76.2	.585	.579	.585	+1.8	.219
ARITH	76.7	78.0	74.4	.950	.895	.950	+1.3	.453

- ULL is the only held-out type with $p < 0.01$
- For Qwen, random control vectors* are always either neutral or harmful; this shows that the steering vector is directionally specific

Paper Table I; *to rule out noise, we used random direction control vectors that have the same magnitude (same L2 norm) and are almost orthogonal to the steering direction

In our cross-model results, 3 out of 6 models show positive and significant steering effects while Llama-8B is the sole anomaly

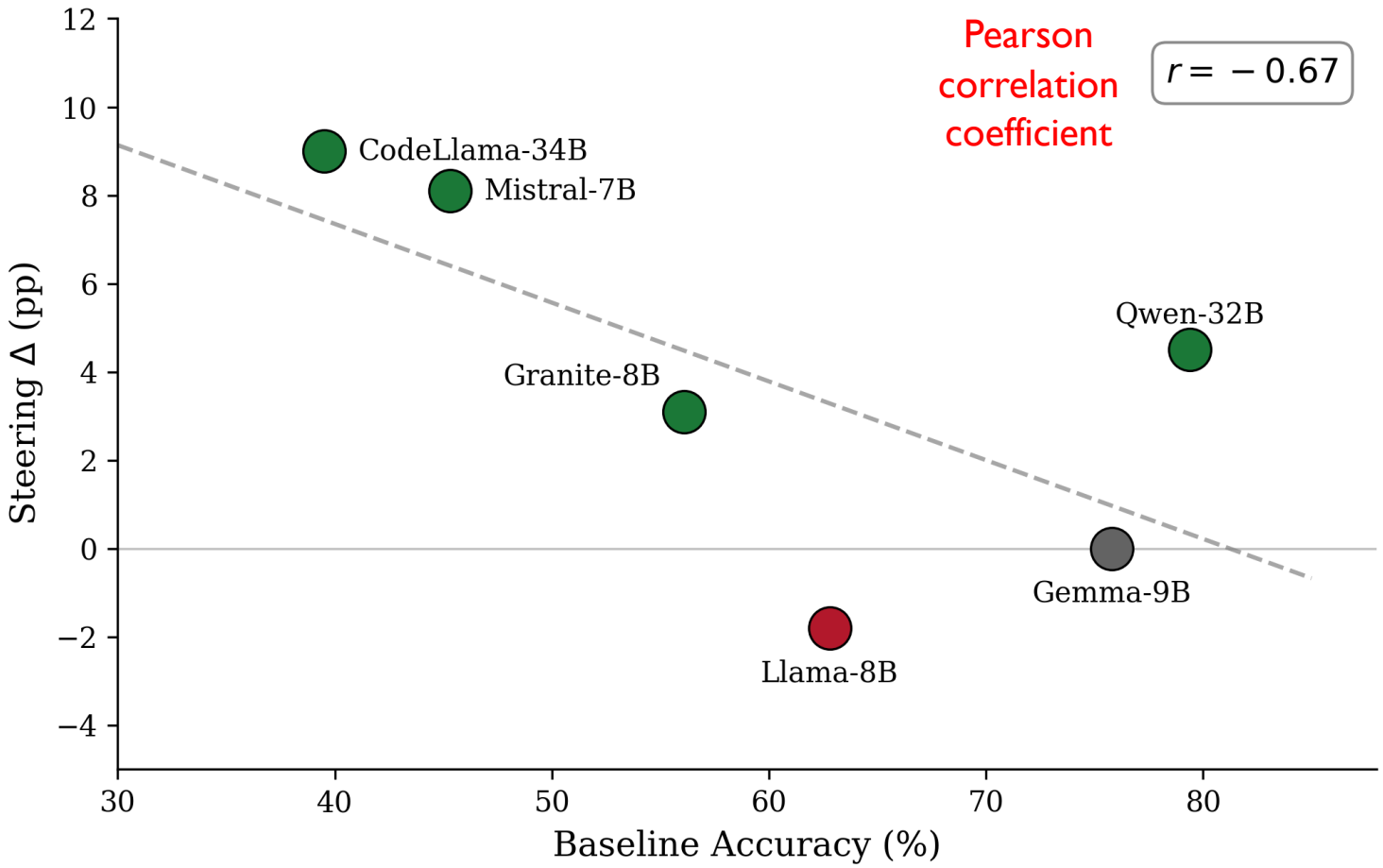
TABLE II

CROSS-MODEL RESULTS ON ULL HELD-OUT ($n = 223$). Δ_S: STEERING IMPROVEMENT. Δ_R: RANDOM DIRECTION CHANGE.

Model	Params	Base	Steered	Δ_S	Δ_R	p
Qwen-32B	32B	79.4	**83.9**	+4.5	−0.9	**0.006**
Gemma-9B	9B	75.8	75.8	+0.0	−4.0	—
Llama-8B	8B	62.8	61.0	−1.8	+6.7	—
Granite-8B	8B	56.1	59.2	+3.1	−11.2	0.296
Mistral-7B	7B	45.3	**53.4**	+8.1	−13.5	**0.018**
CL-34B	34B	39.5	**48.4**	+9.0	−2.2	**0.004**

Paper Table II

Steering helps least where the model is strongest



- The Pearson correlation (r) between baseline accuracy and steering delta is r = −0.67 => strong negative correlation
- This suggests, although not significantly at six models (p≈0.14) that stronger models benefit less from steering.
- Models close to their knowledge ceiling (like Qwen at 79.4%) improve little, while weaker models (like CodeLlama at 39.5%, Mistral at 45.3%) improve by a more significant amount.
- Steering activates what pre-training encoded => it doesn't introduce new concepts

A comparison between lightweight interventions

TABLE III
LIGHTWEIGHT INTERVENTIONS ON ULL HELD-OUT ($n = 223$).
PRECISION AND F1 ARE MACRO-AVERAGED ACROSS ALL 11 CLASSES.

Intervention	Acc	Prec	F1	ULL Rec	ΔAcc
Zero-shot	50.7	47.9	38.8	48.6	−28.7
Few-shot	79.4	69.9	62.7	80.0	—
Few-shot + Steer	**83.9**	**71.1**	**63.5**	**87.1**	**+4.5**

Paper Table III. Macro-averaged precision / F1 across 11 classes on ULL held-out.

CPS is the methodologically more accurate, but it performs poorly; this is consistent with our hypothesis that steering adjusts a decision threshold rather than adding vulnerability reasoning*

CDM (coarse method)	CPS (fine method)
Unpaired group means over cross-domain (across categories) Slither data	Paired per-sample subtraction on domain-matched SmartBugs pairs
Category vectors nearly parallel — cosine 0.75 – 0.91	Category vectors near-orthogonal — cosine 0.06 – 0.09
ULL accuracy 83.9 % (+4.5 pp), nominal p = 0.006	Statistically insignificant

- Across all configurations, Cohen's h stays at or below 0.18 (h=0.20 is the threshold for a small effect); which is consistent with a calibration-scale intervention as opposed to a change in reasoning

*this is an indirect inference; calibration plots and confusion matrices remain future work

Primary contributions

1	First activation-steering study on Solidity — six models, five architectural families, 11-class classification.
2	+4.5 pp on unchecked low-level calls: the only lightweight intervention that improves on few-shot prompting.
3	CDM beats CPS, and the gain falls as baseline accuracy rise; steering calibrates a threshold, it does not teach concepts
4	Hypothetical pre-training ceiling ($r = -0.67$ between baseline accuracy and steering gain)

Department of COMPUTER SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS

Limitations

- The Llama-3.1-8B anomaly indicates that directional specificity may not universally hold across all architectures
- Linear steering only: non-linear concepts may need higher-dimensional techniques
- Although plausible, our calibration interpretation is inferred indirectly, and direct evidence such as confidence-calibration plots and confusion matrices remain future work.

Future Work

PID steering[1]	Feedback-controlled magnitude: addresses α-overshoot in high-α regimes
Gradient-based (COLD-Steer)[2]	Break past the pre-training ceiling with lightweight gradient updates
Non-linear probes[3]	Kernel or MLP probes to test whether certain vulnerabilities are non-linearly encoded

1. D.V. Nguyen et. al, "Activation Steering with a Feedback Controller", arXiv:2510.04309, 2026
2. K. Sharma et. al., "COLD-Steer: Steering Large Language Models via In-Context One-step Learning Dynamics", arXiv:2603.06495, 2026
3. J. Braun, "Understanding Unreliability of Steering Vectors in Language Models: Geometric Predictors and the Limits of Linear Approximations", arXiv:2602.17881, 2026

Thank You!

Questions?

Department of
COMPUTER
SCIENCE
THE UNIVERSITY OF TEXAS AT DALLAS